Recitation Week 10

Hi, my name is Greed and I am one of the emotions that reside in your brain. I love money. Who says money can't buy happiness? In fact, I have infinite amount of money. Unfortunately, I only have coins in different denominations. I often wonder... what is the best way to pay people in coins? Can you help me to spend some of my money?

Consider the problem of making changes of n cents using the fewest number of coins. The coins come from $m+1$ different denominations $d_0, \ldots, d_m$.

1. Consider an optimal solution to the problem of making change for n cents. Let $C(n)$ be the optimal number of coins to make change for n cents. Select an arbitrary coin from the optimal solution. Assume that the denomination of the coin is d_k. Argue that $C(n) = 1 + C(n - d_k)$.

   Note that this implies that if we are able to select and remove a coin that belongs to the optimal solution, we are left with a subproblem of optimally making change for remaining amount (optimal substructure).

   Solution: After removing one coin of denomination d_k, we have one fewer coin in the set that makes up $n - d_k$ cents. This set must contain the optimal number of coins for changing $n - d_k$ cents. Otherwise, we can just replace the the set with a better solution and obtain a contradiction to the optimality of $C(n)$ when we add the coin of d_k back.

2. Suppose that the available coins are in denominations that are powers of c, i.e. the denominations are $c^0, c^1, c^2, \cdots, c^m$ for $c \geq 2$. We first prove a fact that is useful for later showing the optimality of our algorithm: In the optimal solution, the number of coins in each denomination c^k for $k = 0, 1, \ldots, m-1$ (all except the largest denomination) is at most $c - 1$.

   Solution: Assume that we have c or more coins of denomination c^k in the optimal solution. Then we can replace c copies of the coins with one coin of denomination $c^{k+1} = c \times c^k$, reducing the number of coins required and contradicting the optimality of the solution.

3. Now show the following: If $c^k \leq n \leq c^{k+1}$ for $k < m$, the optimal solution must contain a coin from denomination c^k. Also, show that if $n \geq c^m$, the optimal solution must contain a coin from denomination c^m.

   (Hint: Using the result in the previous part, what is the largest number that can be made up using coins from the optimal solution that comes from denominations $c^0, \ldots, c^k$.)

   Solution: Suppose $c^k \leq n < c^{k+1}$ for $0 \leq k < m$. If c^k is not included in the optimal solution for n, then the maximum possible value that can be attained using coins of smaller denomination is

   $$(c-1)(1 + c + c^2 + \cdots + c^{k-1}) = c^k - 1 < c^k$$

   which is a contradiction. Hence, c^k must be included in the optimal solution for n. The same argument also hold for $n \geq c^m$: if there are no coins of denomination c^m the maximum value that can be obtained is less than n.

   Comment: The sum can be derived from the formula for the sum of geometric series. Alternatively, note that $(c-1)(1 + c + c^2 + \cdots + c^{k-1})$ is a telescoping sum $c^k - c^{k-1} + c^k - c^k + \cdots + -c + c - 1 = c^k - 1$.

4. We have shown a greedy choice property that corresponds to the optimal substructure property for this problem, i.e. the optimal solution always contains a coin of the largest denomination that is less than or equal to n.

   Using the result, describe a greedy algorithm to optimally making change for n cents from denomination $c^0, \ldots, c^m$. Give pseudo-code.

   Solution: We describe the algorithm using the algorithm below: Assume that the denominations are in the array $D[0, .., m] = [c^0, \ldots, c^m]$.

```
GREEDY-COIN-CHANGE(n, k)
if n == 0
    return ∅
elseif n < D[k]
    return GREEDY-COIN-CHANGE(n, k − 1)
else return GREEDY-COIN-CHANGE(n − D[k], k) ∪ {D[k]}
```

   To obtain the solution, run the algorithm with $k = m$. Note that it is also possible to add $\lfloor \frac{n}{D[k]} \rfloor$ coins in one go, but the runtime will not be asymptotically better as we still have to add each coin one by one into the output list.